Aurora Drumond Costa Magalhães
Ana Clara de Souza
Kayke Wellington Ribeiro Santos

Especificação e Projeto de Software: Plataforma "Preço Bão"

Timóteo - MG
2026

Aurora Drumond Costa Magalhães
Ana Clara de Souza
Kayke Wellington Ribeiro Santos

Especificação e Projeto de Software: Plataforma "Preço Bão"

Documentação completa de especificação, modelagem e arquitetura de software apresentada ao curso de Engenharia de Software do Centro Universitário Católica do Leste de Minas Gerais (Unileste).

Centro Universitário Católica do Leste de Minas Gerais
Campus Coronel Fabriciano

Timóteo - MG
2026

Resumo

O presente trabalho documenta a especificação e o projeto da plataforma “Preço Bão”, destinada à pesquisa, comparação e acompanhamento de preços de medicamentos e produtos de farmácia. O objetivo é relacionar os requisitos do sistema aos artefatos de modelagem, persistência, interface e planejamento de testes. O procedimento de consolidação consiste na leitura dos documentos existentes e na conferência de modelos, contratos de dados, rotas, serviços e configurações do repositório, distinguindo funcionalidades previstas de componentes presentes no código. A implementação examinada organiza um backend FastAPI e um frontend React Native com Expo em um monorepo, com persistência PostgreSQL e coleta de ofertas em fontes públicas. O documento apresenta minimundo, requisitos funcionais e não funcionais, casos de uso, classes, diagrama entidade-relacionamento, dicionário de dados, protótipo de interface e plano de testes. As Práticas 1 e 2 do oitavo período acrescentam análise de riscos de manutenção, alternativas de intervenção, backlog e plano coletivo de sincronização de favoritos, registrados como propostas. A conferência identifica componentes de autenticação, favoritos, catálogo e histórico, além de limitações em recuperação de senha, geolocalização, retenção de dados e automação de qualidade. Os casos do plano permanecem pendentes, sem atribuição de resultados de execução. Conclui-se que a documentação deve preservar os objetivos aprovados e explicitar as diferenças entre planejamento e implementação, sem tomar a existência do código como comprovação de validação operacional ou de conformidade normativa.

Palavras-chave: engenharia de software; especificação de requisitos; *web scraping*; modelagem de sistemas; arquitetura de *software*.

Abstract

This work documents the specification and design of the “Preço Bão” platform, intended for searching, comparing, and tracking prices of medicines and pharmacy products. Its objective is to relate system requirements to modeling, persistence, interface, and test planning artifacts. The consolidation procedure consists of reading existing documents and examining models, data contracts, routes, services, and repository configurations, distinguishing planned features from components present in the code. The implementation examined organizes a FastAPI backend and a React Native frontend with Expo in a monorepo, with PostgreSQL persistence and offer collection from public sources. The document presents the domain description, functional and nonfunctional requirements, use cases, classes, entity-relationship diagram, data dictionary, interface prototype, and test plan. Practices 1 and 2 from the eighth academic term add maintenance risk analysis, intervention alternatives, a backlog, and a collective favorites synchronization plan, recorded as proposals. The examination identifies authentication, favorites, catalog, and history components, as well as limitations in password recovery, geolocation, data retention, and quality automation. Test plan cases remain pending, with no execution results attributed to them. The work concludes that documentation should preserve approved objectives and explain the differences between planning and implementation, without treating the existence of code as evidence of operational validation or regulatory compliance.

Keywords: software engineering; requirements specification; web scraping; systems modeling; software architecture.

Lista de ilustrações

Figura 1 – Casos de uso do escopo previsto da plataforma 13
Figura 2 – Classes de coleta e módulo funcional Indiana 15
Figura 3 – Famílias de contratos de dados da API 16
Figura 4 – Visão resumida das classes ORM e associações 17
Figura 5 – DER do esquema ORM para PostgreSQL 18
Figura 6 – Visão geral do protótipo de baixa fidelidade 24
Figura 7 – Ampliação do protótipo: login e início 25
Figura 8 – Ampliação do protótipo: detalhe e pesquisa 26
Figura 9 – Ampliação do protótipo: histórico de preços 27

Lista de quadros

Quadro 1 – Especificação dos Requisitos Funcionais 10
Quadro 2 – Requisitos Não Funcionais: Arquitetura e Performance 10
Quadro 3 – Requisitos Não Funcionais: Segurança e Conformidade 11
Quadro 4 – Requisitos Não Funcionais: Engenharia e Qualidade de Código 11
Quadro 5 – Dicionário: anvisa_medicamentos 19
Quadro 6 – Dicionário: catalogo_base 20
Quadro 7 – Dicionário: farmacias 21
Quadro 8 – Dicionário: usuario 22
Quadro 9 – Dicionário: ofertas_farmacia 22
Quadro 10 – Dicionário: usuario_farmacia_favorita 23
Quadro 11 – Dicionário: usuario_medicamento_favorito 23
Quadro 12 – Dicionário: historico_precos 23
Quadro 13 – Requisitos funcionais selecionados para teste 30
Quadro 14 – Requisitos não funcionais selecionados para teste 30
Quadro 15 – Estratégias de teste propostas 31
Quadro 16 – Casos de teste da plataforma Preço Bão - Prioridade Alta 32
Quadro 17 – Casos de teste da plataforma Preço Bão - Prioridade Média 33
Quadro 18 – Casos de teste da plataforma Preço Bão - Prioridade Baixa 33
Quadro 19 – Equipe e responsabilidades 34
Quadro 20 – Cronograma simplificado de testes 34
Quadro 21 – Hipóteses de defeitos para investigação 35
Quadro 22 – Riscos relacionados ao projeto e aos testes 35
Quadro 23 – Itens de manutenção propostos na Prática 1 39
Quadro 24 – Estimativas propostas na Prática 1 39
Quadro 25 – Demandas e classificação de manutenção da Prática 2 41
Quadro 26 – Backlog de manutenção proposto na Prática 2 42
Quadro 27 – Sprints planejadas na Prática 2 44

Sumário

1 INTRODUÇÃO 8
1.1 Contexto e procedimento de análise documental 8

2 DESENVOLVIMENTO 9
2.1 Minimundo 9
2.2 Especificação de Requisitos 9
2.2.1 Situação dos requisitos na implementação 11
2.3 Modelagem do Sistema 12
2.3.1 Diagrama de Casos de Uso 13
2.3.2 Diagrama de Classes 14
2.3.2.1 Motor de *scraping* (ETL) 14
2.3.2.2 *Schemas* da API (Pydantic) 15
2.3.2.3 Mapeamento objeto-relacional (SQLAlchemy ORM) 15
2.4 Projeto de banco de dados 17
2.4.1 Restrições e divergências entre fontes 19
2.4.2 Dicionário de dados 19
2.4.2.1 anvisa_medicamentos 19
2.4.2.2 catalogo_base 20
2.4.2.3 farmacias 21
2.4.2.4 usuario 21
2.4.2.5 ofertas_farmacia 22
2.4.2.6 usuario_farmacia_favorita 23
2.4.2.7 usuario_medicamento_favorito 23
2.4.2.8 historico_precos 23
2.5 Projeto de interfaces e navegação 24
2.6 Plano de Testes 28
2.6.1 Introdução 28
2.6.2 Escopo dos Testes 28
2.6.3 Requisitos a Serem Testados 30
2.6.4 Estratégia de Testes 30
2.6.5 Casos de Teste 32
2.6.6 Infraestrutura e Recursos de Teste 33
2.6.7 Equipe e Responsabilidades 34
2.6.8 Cronograma de Testes 34
2.6.9 Registro de Defeitos e Riscos 35

3 CONFIGURAÇÃO E MANUTENÇÃO DE SOFTWARE 37
3.1 Prática 1: alternativas para a manutenção do sistema 37
3.1.1 Solicitação e produto elaborado 37
3.1.2 Síntese da leitura e alternativas selecionadas 37
3.1.3 Componentes e dificuldades retomados 38
3.1.4 Problemas potenciais e propostas de manutenção 38
3.1.5 Síntese da proposta 40
3.2 Prática 2: identificação e planejamento de manutenções 40
3.2.1 Solicitação, classificação e produto elaborado 40
3.2.2 Etapas 1 e 2: levantamento, classificação e análise 41
3.2.3 Etapa 3: priorização e backlog 42
3.2.4 Etapa 4: plano coletivo de ação para M-01 42
3.2.5 Etapa 5: cronograma de sprints 43
3.2.6 Etapa 6: reflexão e indicadores propostos 44

4 CONCLUSÃO 45

REFERÊNCIAS 46

1 Introdução

O projeto “Preço Bão” propõe uma plataforma móvel e web para pesquisa, comparação e acompanhamento de preços de medicamentos e produtos de farmácia. A proposta reúne ofertas de fontes públicas em um catálogo centralizado, permitindo consultar preços e acompanhar produtos de interesse. A informação coletada representa uma observação da fonte; não confirma disponibilidade física nem substitui a consulta ao estabelecimento.

O objetivo desta documentação é apresentar a especificação e os artefatos de projeto e confrontá-los com a implementação presente no repositório. São mantidos os identificadores RF01–RF08 e RNF01–RNF11, inclusive quando descrevem metas ainda não demonstradas. A existência de um componente no código não equivale à comprovação de seu funcionamento em produção.

1.1 Contexto e procedimento de análise documental

O repositório separa a API em Python/FastAPI, o aplicativo React Native/Expo e os recursos de configuração do ambiente. A análise documental utiliza os fontes LaTeX existentes, os modelos SQLAlchemy, os contratos Pydantic, as rotas da API, os serviços de coleta e as configurações versionadas como evidência da estrutura técnica. O dicionário de dados é derivado dos metadados ORM para PostgreSQL e confrontado com a cadeia de migrações disponível. Esse procedimento descreve a conferência dos artefatos e não caracteriza, por inferência, o método de pesquisa ou o processo histórico adotado pela equipe.

Os casos de teste são apresentados como planejamento, com seus estados pendentes preservados. Testes automatizados e relatórios técnicos existentes são fontes distintas do plano acadêmico: sua presença não autoriza atribuir execução, participação individual ou cumprimento de atividades letivas sem o registro correspondente.

O Capítulo 2 apresenta minimundo, requisitos, modelagem, banco de dados, interfaces e plano de testes. O Capítulo 3 reúne as respostas documentadas das Práticas 1 e 2 do oitavo período, com alternativas de manutenção, classificação de demandas, backlog e plano coletivo de ação. Esses produtos constituem análise e planejamento; não se atribui execução às propostas. A conclusão sintetiza os componentes identificados e as limitações de validação. A atribuição de artefatos a períodos letivos requer os respectivos enunciados e registros acadêmicos.

2 Desenvolvimento

Este capítulo tem como objetivo detalhar a concepção e a estruturação do projeto "Preço Bão". Nas seções a seguir, serão apresentados os elementos fundamentais para a compreensão do escopo do sistema, iniciando pela delimitação do minimundo, que descreve o domínio do problema, os fluxos operacionais e os principais requisitos que guiarão a modelagem do banco de dados e a arquitetura do software.

2.1 Minimundo

O projeto consiste em uma plataforma de serviço, acessível via aplicativo móvel e site, voltada para ajudar o consumidor final a pesquisar, comparar e monitorar os preços de medicamentos e produtos de farmácia.

A arquitetura prevê a alimentação por rotinas automatizadas de extração de dados (*web scraping*[1]) que varre os sites de diversas farmácias da região. Durante esse processo, o sistema coleta, processa e armazena informações vitais, como a disponibilidade dos produtos, os laboratórios fabricantes e os valores cobrados, construindo um grande catálogo centralizado de remédios.

Através da interface do aplicativo, o usuário final pode pesquisar pelos medicamentos de uso contínuo ou produtos que precisa comprar. A interface implementada apresenta os produtos em cartões de exibição, listando as informações do laboratório e comparando os melhores preços encontrados no mercado. Como funcionalidade prevista (RF06), o aplicativo deverá receber coordenadas mediante permissão e calcular distâncias até os estabelecimentos. Esse fluxo não está integrado às rotas de pesquisa consultadas; coordenadas de farmácias no modelo não comprovam geolocalização do consumidor.

O código contém favoritos vinculados à conta e consulta de observações de preço por produto e farmácia. Esses registros auxiliam o acompanhamento de variações, sem garantir o melhor momento ou local de compra. Preços, disponibilidade e cobertura dependem das fontes e dos instantes de coleta.

2.2 Especificação de Requisitos

Nesta seção, os requisitos do sistema "Preço Bão" são detalhados e classificados. Os Requisitos Funcionais (RF) descrevem as ações e comportamentos que o sistema deve executar para atender às necessidades do usuário, enquanto os Requisitos Não Funcionais

[1] Técnica computacional automatizada em que um *script* ou algoritmo navega por páginas da web com o objetivo de raspar, coletar e estruturar em um banco de dados as informações públicas que originalmente são exibidas de forma não estruturada em código HTML.

(RNF) especificam os critérios de qualidade, restrições tecnológicas e de desempenho que o sistema deve respeitar.

Quadro 1 – Especificação dos Requisitos Funcionais

ID	Descrição do Requisito
RF01	**Extração Resiliente de Dados:** O sistema deve possuir uma rotina automatizada de *web scraping* para coletar dados de farmácias. Em caso de falha na extração, o sistema deve registrar o erro em *log* e continuar a varredura nas demais fontes.
RF02	**Processamento e Catálogo:** O sistema deve armazenar as informações vitais extraídas e enriquecê-las contra a base da CMED/ANVISA. A disponibilidade em estoque deve ser exibida com um aviso claro de que é uma "estimativa sujeita a confirmação no local".
RF03	**Gestão de Usuários:** O aplicativo deve permitir o cadastro de novos usuários, autenticação (*login*) e recuperação de senha, criando um perfil individual.
RF04	**Pesquisa e Neutralidade Algorítmica:** O consumidor deve poder pesquisar por medicamentos. O algoritmo de ordenação deve garantir a neutralidade, priorizando estritamente o menor preço ou a menor distância.
RF05	**Exibição em Cartões:** O sistema deve exibir os resultados em cartões padronizados, listando informações do laboratório e comparando os melhores preços.
RF06	**Cálculo de Distância Geográfica:** Mediante autorização prévia, o aplicativo deve receber as coordenadas geográficas do usuário e calcular a distância até os estabelecimentos, ordenando por proximidade.
RF07	**Salvamento de Produtos:** O sistema deve permitir que um usuário autenticado favorite medicamentos de uso contínuo em sua conta.
RF08	**Monitoramento de Preços:** O sistema deve gerar e exibir um gráfico ou histórico rastreando a flutuação de preços dos remédios salvos ao longo das últimas semanas.

Fonte: Elaborado pelos autores (2026).

Quadro 2 – Requisitos Não Funcionais: Arquitetura e Performance

ID	Descrição do Requisito
RNF01	**Performance da API:** O *backend* (Python/FastAPI) deve ser assíncrono e responder a requisições de pesquisa em um tempo máximo de 500 milissegundos no percentil 95.
RNF02	**Arquitetura do Frontend:** O aplicativo móvel deve ser desenvolvido com React Native e Expo utilizando TypeScript, com a meta de obter *builds* funcionais para Android e iOS.
RNF03	**Persistência e Retenção:** Os dados devem ser armazenados em um PostgreSQL. O sistema deve implementar uma política de limpeza (*data purging*), excluindo históricos com mais de 12 meses.

Fonte: Elaborado pelos autores (2026).

Quadro 3 – Requisitos Não Funcionais: Segurança e Conformidade

ID	Descrição do Requisito
RNF04	**Segurança e Criptografia:** Todas as senhas devem ser armazenadas utilizando *hashing* fortes (ex: bcrypt). A comunicação deve ocorrer obrigatoriamente sob protocolo HTTPS/TLS.
RNF05	**Privacidade (LGPD):** O sistema não deve armazenar o histórico contínuo de localização do usuário. As coordenadas devem ser processadas em memória e descartadas em seguida.

Fonte: Elaborado pelos autores (2026).

Quadro 4 – Requisitos Não Funcionais: Engenharia e Qualidade de Código

ID	Descrição do Requisito
RNF06	**Estrutura e Validação:** O sistema deve utilizar ORM e Pydantic no *backend* para validação. O TypeScript no *frontend* deve usar modo estrito (`strict: true`).
RNF07	**Organização em Monorepo:** O projeto deve adotar a estrutura de monorepo, isolando as dependências do *frontend* e do *backend*.
RNF08	**Orquestração Local:** O ambiente de desenvolvimento deve ser provisionado via `docker-compose.yml`.
RNF09	**Automação de Comandos:** O repositório deve conter um `Makefile` na raiz para abstrair comandos complexos (ex.: `make db-up`).
RNF10	**CI/CD e Qualidade:** Fluxos do GitHub Actions devem bloquear *Pull Requests* que falhem nos testes ou na análise estática (ESLint/Ruff).
RNF11	**Padronização de Estilo:** O repositório deve impor formatação via `.editorconfig` e possuir um `.gitignore` configurado para excluir arquivos sensíveis do versionamento.

Fonte: Elaborado pelos autores (2026).

2.2.1 Situação dos requisitos na implementação

Os Quadros 1 a 4 expressam objetivos do projeto. A conferência estática dos fontes distingue presença de componentes de validação por execução. Os testes do plano permanecem pendentes.

- **RF01–RF02:** há coleta em `app/scrapers/`, persistência e enriquecimento local em `app/services/enriquecimento.py`. O ETL CMED é acionável por comandos; isso não prova agendamento periódico nem cobertura de todas as fontes. O aviso de estoque estimado exige conferência da interface e não decorre de um campo booleano.

- **RF03:** as rotas em `app/api/routes/usuarios.py` integram cadastro, autenticação e perfil. Os callbacks de recuperação em `app/core/auth.py` apenas registram a solicitação; não há inclusão do roteador de recuperação nem entrega de token demonstrada no fluxo público.

- **RF04–RF06:** há pesquisa e ordenação por preço, nome e relevância em `app/api/routes/catalogo.py`, além de cartões no frontend. A menor distância permanece prevista. Não se infere neutralidade completa da existência de uma ordenação: critérios e resultados precisam de teste.

- **RF07–RF08:** favoritos possuem vínculos persistentes por usuário e verificação de autenticação. A consulta de histórico em `app/api/routes/historico.py` agrupa observações por farmácia e período; é pública e não exige que o produto esteja favoritado. A persistência é implementada em `app/services/preco_observacoes.py`; o gráfico é composto por `HistoryChart.tsx`.

Quanto aos requisitos não funcionais, FastAPI e rotinas assíncronas estão presentes, mas o acesso SQLAlchemy usa sessões síncronas; não se afirma atendimento integral ao RNF01 nem ao p95 de 500 ms sem medição. React Native, Expo e TypeScript constam dos fontes (RNF02); exportação não comprova fluxos funcionais Android e iOS. PostgreSQL está configurado, mas filtros temporais do histórico não constituem exclusão automática após 12 meses (RNF03).

A autenticação utiliza a infraestrutura FastAPI Users e armazenamento em `hashed_password`. O Compose contém túnel para a API, mas não comprova HTTPS/TLS em todos os ambientes; o coletor Indiana contém cliente com verificação de certificado desativada (RNF04). A ausência de histórico contínuo de localização no modelo não comprova conformidade integral à LGPD nem o fluxo de descarte ainda não implementado (RNF05).

ORM, Pydantic, configuração TypeScript estrita, organização em monorepo, Compose e Makefile sustentam os componentes de RNF06–RNF09. O alvo real de provisionamento do banco é `make db-up`. Não foram localizados workflows em `.github/workflows/`; scripts locais de análise não comprovam bloqueio de PRs nem implantação contínua (RNF10). `.editorconfig` existe, mas está vazio e não impõe regras; `.gitignore` exclui arquivos sensíveis comuns (RNF11). Esses arquivos não garantem ausência de segredos em todo o histórico.

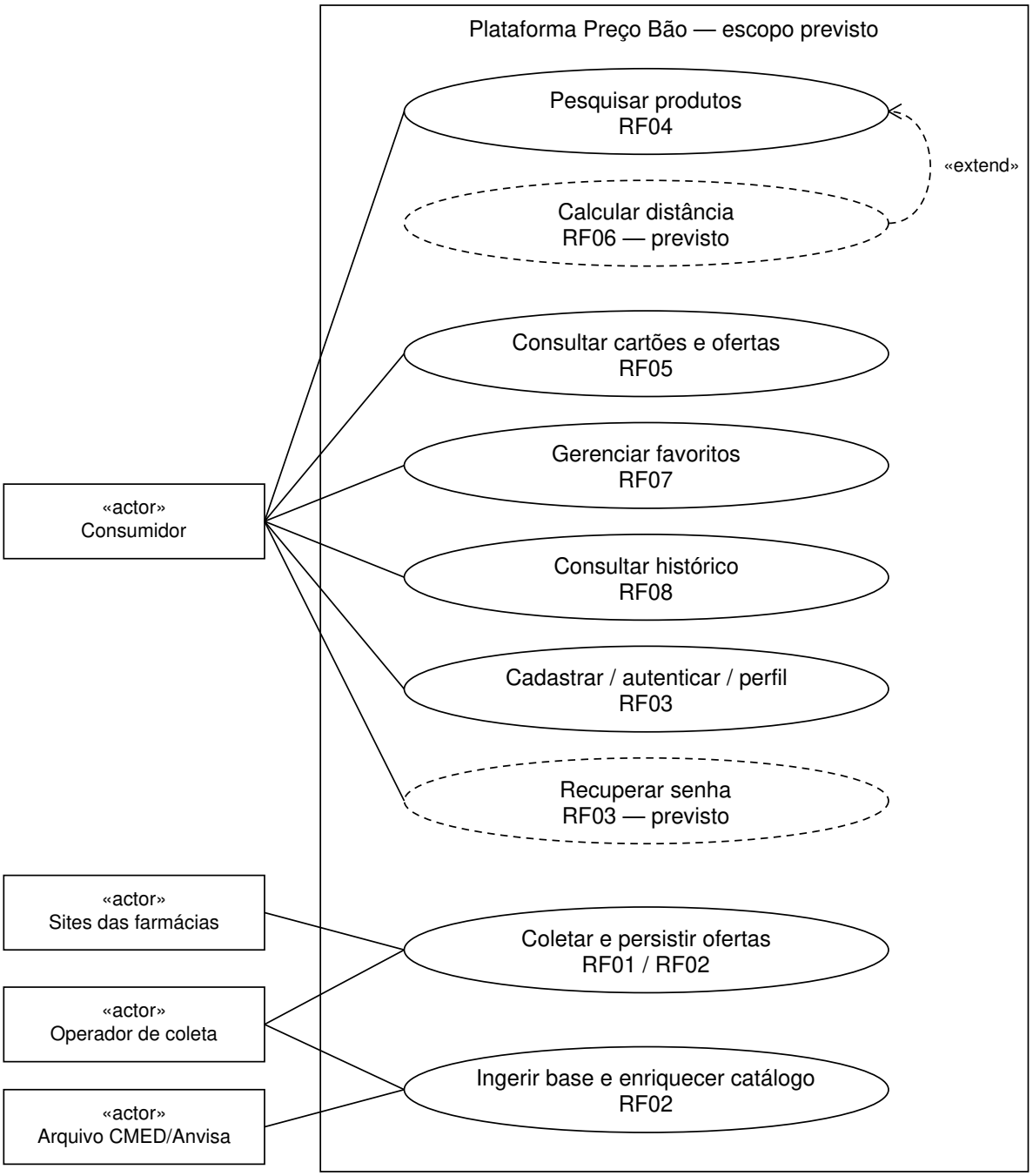
2.3 Modelagem do Sistema

A modelagem usa a UML para relacionar o comportamento esperado aos componentes. A abordagem de casos de uso é descrita por Booch, Rumbaugh e Jacobson (2005); a organização de modelos de domínio e projetos de classes é discutida por Larman (2007). Os diagramas a seguir representam o escopo planejado ou a estrutura técnica, conforme indicado em cada figura.

2.3.1 Diagrama de Casos de Uso

A Figura 1 delimita a plataforma e relaciona o consumidor aos serviços de pesquisa, comparação, conta, favoritos e histórico. O operador aciona coleta e ingestão; sites de farmácias e arquivo CMED são fontes externas. O robô interno não é modelado como ator externo da própria plataforma.

Figura 1 – Casos de uso do escopo previsto da plataforma



Fonte: Elaboração a partir dos requisitos e das rotas do projeto (2026).

Autenticação é precondição para alterar perfil e gerenciar favoritos, sem exigir repeti-

ção do login a cada operação. Por isso esses casos não incluem automaticamente o caso de autenticar. O histórico implementado é público. O cálculo de distância é uma extensão prevista da pesquisa no ponto de apresentação/ordenação dos resultados, condicionada à permissão e à disponibilidade das coordenadas. Não há relação de inclusão artificial entre consulta e comparação, que podem ser realizadas separadamente.

UC01 – Coletar ofertas (RF01). Operador inicia a rotina; o coletor consulta a fonte e persiste observações. Falhas devem ser registradas e tratadas; continuidade entre fontes requer CT12.

UC02 – Enriquecer catálogo (RF02). Operador fornece a base oficial; a aplicação cruza EANs. Ausência de correspondência não deve ser apresentada como validação regulatória concluída.

UC03 – Gerenciar conta (RF03). Consumidor cadastra-se e autentica-se para acessar perfil. Dados inválidos devem ser rejeitados. Recuperação de senha é uma meta sem fluxo público demonstrado.

UC04 – Pesquisar ofertas (RF04/RF05). Consumidor informa uma busca, consulta cartões e ofertas e escolhe ordenação. Sem resultado, a interface informa a ausência de itens.

UC05 – Calcular distância (RF06). Extensão prevista de UC04, mediante permissão e coordenadas. Sem permissão, a pesquisa por preço deve continuar disponível.

UC06 – Gerenciar favoritos (RF07). Conta ativa autenticada adiciona ou remove um produto; o vínculo é persistido para aquele usuário. Acesso sem autenticação deve ser recusado.

UC07 – Consultar histórico (RF08). Consumidor consulta produto e período; a API retorna observações por farmácia e a interface apresenta a série. Uma série vazia não autoriza criar preços ou datas fictícios.

2.3.2 Diagrama de Classes

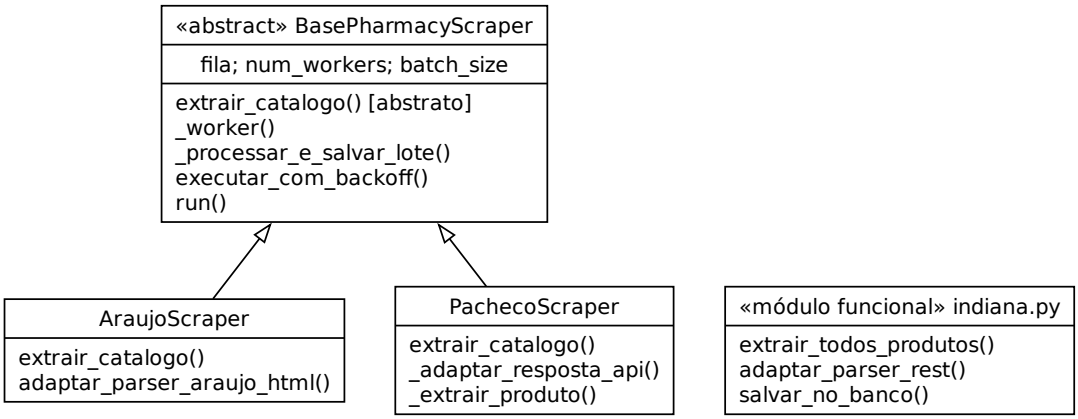
As três famílias distinguem coleta, contratos de dados e persistência. Os desenhos resumem membros relevantes; a enumeração completa das colunas está na Seção 2.4.2.

2.3.2.1 Motor de *scraping* (ETL)

A classe abstrata real é `BasePharmacyScraper`, em `app/scrapers/base.py`. Define `extrair_catalogo` como método abstrato e oferece fila, processamento de lotes, workers e retentativas com backoff. `AraujoScraper` e `PachecoScraper` herdam essa classe. O

módulo Indiana usa funções, incluindo `extrair_todos_produtos`, `adaptar_parser_rest` e `salvar_no_banco`, sem herança da base. A Figura 2 mantém essa distinção.

Figura 2 – Classes de coleta e módulo funcional Indiana



Fonte: Elaboração a partir de `app/scrapers/` (2026).

2.3.2.2 *Schemas* da API (Pydantic)

`UsuarioRead`, `UsuarioCreate` e `UsuarioUpdate` derivam, respectivamente, de `BaseUser`, `BaseUserCreate` e `BaseUserUpdate` da biblioteca FastAPI Users. Leitura não inclui o hash; criação recebe o campo `password` herdado. Não há `UsuarioBase`, `UsuarioOut` ou `senha_clara` nos contratos consultados.

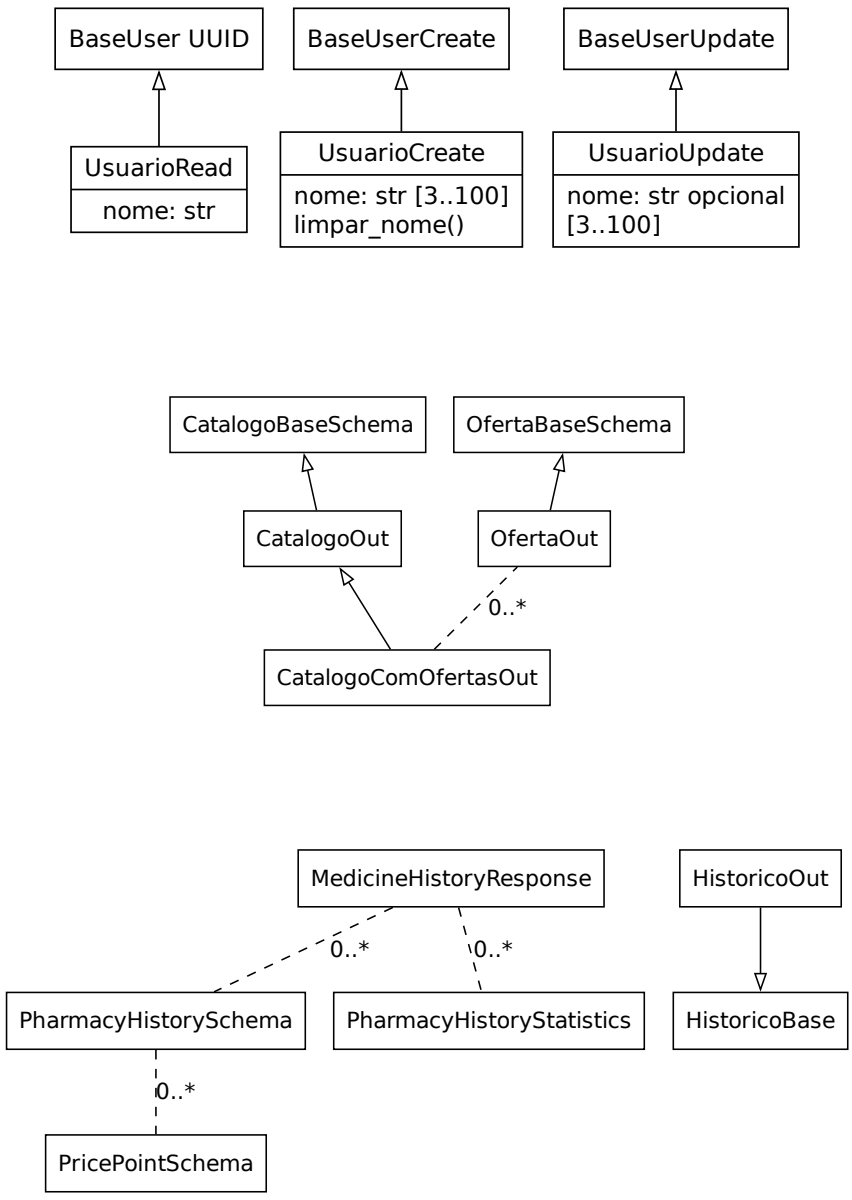
Catálogo e ofertas usam famílias próprias: `CatalogoOut` deriva de `CatalogoBaseSchema`; `CatalogoComOfertasOut` agrega ofertas, sem herdar delas. `OfertaOut` deriva de `OfertaBaseSchema`. Para histórico coexistem contratos legados `HistoricoBase/Out` e a resposta agregada `MedicineHistoryResponse`, com séries e estatísticas. A Figura 3 resume essas relações; linhas tracejadas sem triângulo representam referências a objetos, não herança.

2.3.2.3 Mapeamento objeto-relacional (SQLAlchemy ORM)

`Base` gerencia metadados e convenções; não injeta automaticamente colunas de identificação ou datas em todas as entidades. `Usuario` herda UUID, e-mail, `hashed_password` e indicadores de conta de `SQLAlchemyBaseUserTableUUID`, acrescentando `nome`. Não possui `criado_em` no modelo consultado.

`CatalogoBase` separa a identidade do produto das ofertas comerciais de `OfertaFarmacia`; inclui classificação e metadados de imagem, que podem evoluir. `Farmacia` mantém dados do estabelecimento. `HistoricoPreco` vincula produto e farmácia e, opcionalmente,

Figura 3 – Famílias de contratos de dados da API

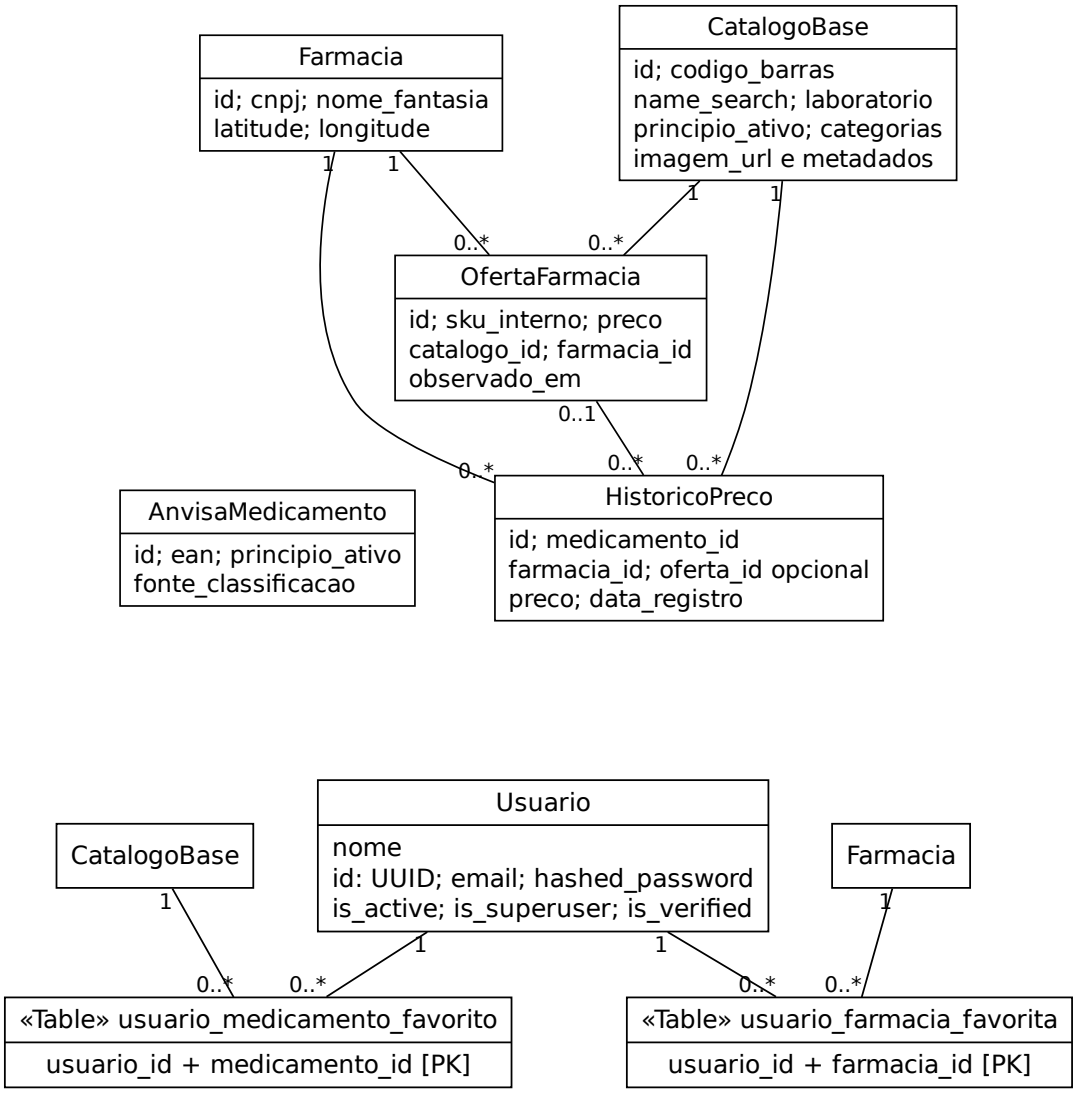


Fonte: Elaboração a partir de `app/schemas/` (2026).

a oferta observada. `AnvisaMedicamento` é uma base local de enriquecimento por EAN; não existe FK entre ela e o catálogo.

Favoritos são duas tabelas associativas, `usuario_medicamento_favorito` e `usuario_farmacia_favorita`, com chaves primárias compostas; não existe classe persistente genérica `Favorito`. A Figura 4 distingue as classes das associações do tipo `Table`.

Figura 4 – Visão resumida das classes ORM e associações



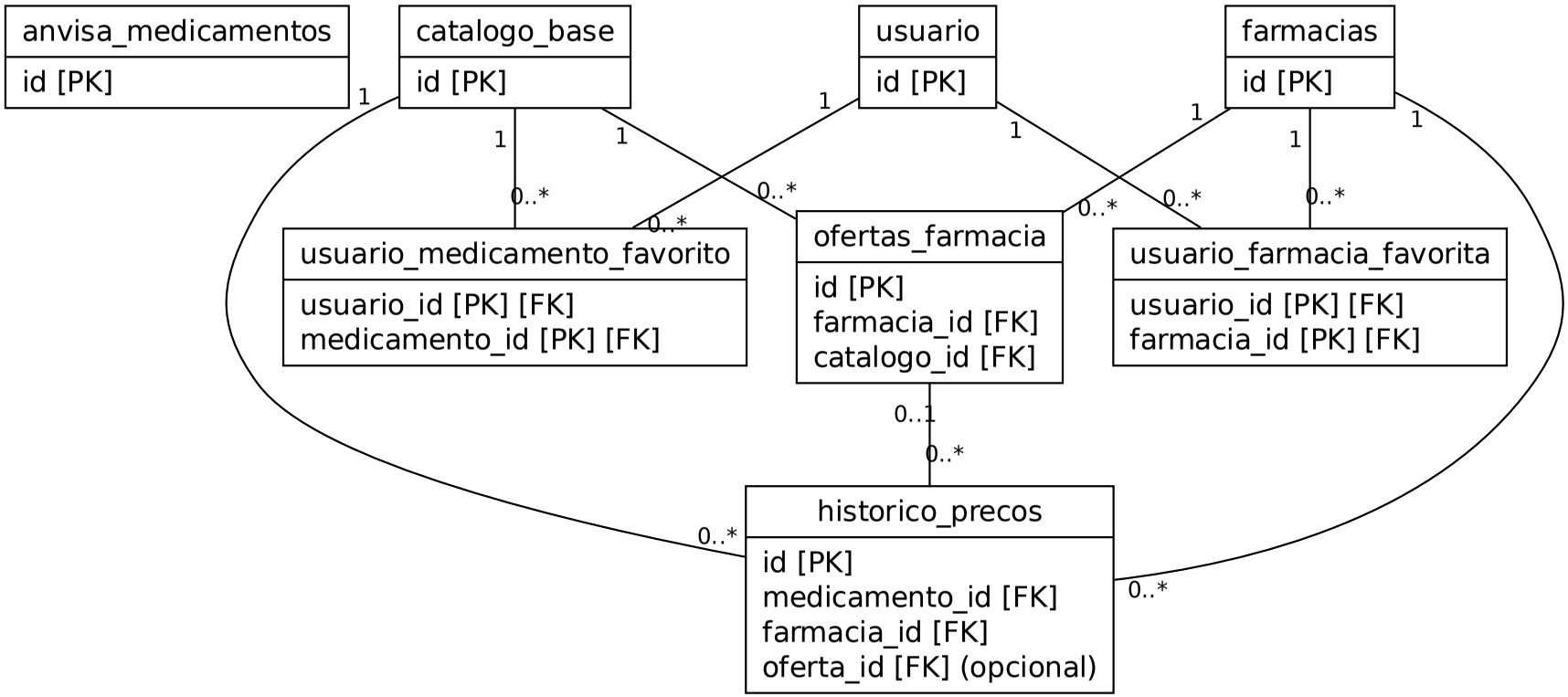
Fonte: Elaboração a partir de `app/models/` (2026).

2.4 Projeto de banco de dados

O esquema documentado é o dos metadados ORM do checkout examinado, compilados para PostgreSQL e confrontados com as migrações até `e4902ba81c6d`. Isso não certifica a versão de um banco em execução. A Figura 5 é o diagrama entidade-relacionamento (DER), com chaves e cardinalidades; o dicionário apresenta todas as colunas.

Cada oferta pertence a um produto e a uma farmácia; cada um pode ter zero ou muitas ofertas. Cada observação histórica pertence a um produto e a uma farmácia e pode referenciar zero ou uma oferta; uma oferta pode ter muitas observações. As associações de favoritos resolvem relações muitos-para-muitos com o usuário. Cada par

Figura 5 – DER do esquema ORM para PostgreSQL



Fonte: Metadados SQLAlchemy em `app/models/` (2026).

de chaves é único pela PK composta, com exclusão em cascata no banco para as duas FKs dessas associações. Cascatas de relacionamento no ORM não devem ser confundidas com `ONDELETE` em outras FKs.

As entidades podem ser localizadas diretamente nas descrições: `anvisa_medicamentos` (2.4.2.1), `catalogo_base` (2.4.2.2), `farmacias` (2.4.2.3), `usuario` (2.4.2.4), `ofertas_farmacia` (2.4.2.5), `historico_precos` (2.4.2.8) e as associações de favoritos (2.4.2.6 e 2.4.2.7).

2.4.1 Restrições e divergências entre fontes

As chaves únicas incluem CNPJ, código de barras quando presente, EAN da base Anvisa, URL de origem da oferta e e-mail. O SKU não é globalmente único. O esquema não declara restrições CHECK para preço positivo ou limites de coordenadas; validações de aplicação não equivalem a restrições no banco. Campos oficiais anuláveis preservam a distinção entre classificação desconhecida e falsa.

A migração `1de74e4279f7` define padrão SQL de array vazio para categorias; o modelo define `default=list` em Python. A migração `3a72d8ce1f40` cria índice composto de histórico por medicamento, farmácia e data, ausente da declaração de índices ORM. A representação ORM é usada no DER e no dicionário, sem omitir essas diferenças. O dicionário inclui padrões SQL declarados no ORM; os padrões apenas Python não se aplicam a inserções SQL diretas. Não se comprova retenção por 12 meses pela existência de datas ou filtros temporais.

2.4.2 Dicionário de dados

A descrição representa os metadados ORM do checkout revisado, compilados para PostgreSQL. Nulo indica se a coluna admite SQL NULL; PK é chave primária e FK é chave estrangeira. Padrões Python são aplicados pela aplicação e não equivalem a padrões SQL. Não há consulta a um banco em execução.

2.4.2.1 anvisa_medicamentos

Base local de consulta oficial; cruzamento por EAN na aplicação, sem FK para o catálogo.

Quadro 5 – Dicionário: anvisa_medicamentos

Coluna	**Tipo / nulo**	**Descrição e restrições**
`id`	INTEGER / não	Identificador do registro. PK.
`ean`	VARCHAR(14) / não	Código EAN da base oficial. único.
`principio_ativo`	VARCHAR / sim	Princípio ativo informado pela fonte.
`laboratorio`	VARCHAR / sim	Laboratório informado.

Continua na próxima página.

Quadro 5 – continuação

Coluna	Tipo / nulo	Descrição e restrições
tarja	VARCHAR / sim	Classificação de tarja informada.
registro_anvisa	VARCHAR(20) / sim	Identificador de registro na Anvisa.
categoria_regulatoria	VARCHAR(40) / sim	Categoria regulatória identificada na fonte.
status_registro	VARCHAR(40) / sim	Situação do registro na fonte.
referencia_anvisa	BOOLEAN / sim	Indicador de medicamento de referência.
exige_receita_confirmada	BOOLEAN / sim	Exigência de receita confirmada na fonte.
controle_especial	BOOLEAN / sim	Indicador de controle especial.
fonte_classificacao	VARCHAR(120) / sim	Procedência da classificação oficial.
atualizado_fonte_em	DATE / sim	Data de atualização informada pela fonte.

Fonte: metadados ORM do projeto (2026).

2.4.2.2 catalogo_base

Identidade e classificação dos produtos e referências de imagens.

Quadro 6 – Dicionário: catalogo_base

Coluna	Tipo / nulo	Descrição e restrições
id	INTEGER / não	Identificador do registro. PK.
codigo_barras	VARCHAR(14) / sim	EAN/GTIN do item; pode estar ausente. único.
name_search	VARCHAR(150) / não	Nome utilizado nas pesquisas.
principio_ativo	VARCHAR(500) / não	Princípio ativo informado pela fonte. padrão Python: `Nãoinformado`.
laboratorio	VARCHAR(100) / não	Laboratório informado. padrão Python: `Nãoinformado`.
exige_receita	BOOLEAN / não	Indicador legado de exigência de receita. padrão Python: `False`.
categoria_regulatoria	VARCHAR(40) / sim	Categoria regulatória identificada na fonte.
referencia_anvisa	BOOLEAN / sim	Indicador de medicamento de referência.
exige_receita_confirmada	BOOLEAN / sim	Exigência de receita confirmada na fonte.
controle_especial	BOOLEAN / sim	Indicador de controle especial.
classificacao_fonte	VARCHAR(80) / sim	Procedência da classificação.
categorias	VARCHAR[] / não	Lista de categorias do catálogo. padrão Python: `list`.
imagem_url	VARCHAR(500) / sim	Referência externa da imagem; não armazena a fotografia.
imagem_fonte	VARCHAR(40) / sim	Origem da referência de imagem.
imagem_sku_origem	VARCHAR(80) / sim	SKU que sustenta a associação da imagem.
imagem_ean	VARCHAR(14) / sim	EAN usado como evidência da imagem.
imagem_match	VARCHAR(24) / sim	Tipo de correspondência de identidade.
imagem_status	VARCHAR(20) / sim	Estado de verificação da referência.

Continua na próxima página.

Quadro 6 – continuação

Coluna	Tipo / nulo	Descrição e restrições
`imagem_atualizada_em`	timestamp com fuso / sim	Instante de alteração da referência.
`imagem_verificada_em`	timestamp com fuso / sim	Última validação bem-sucedida.
`imagem_ultima_verificacao_em`	timestamp com fuso / sim	Instante da última tentativa de validação.
`imagem_falhas_consecutivas`	INTEGER / não	Contador de falhas seguidas. padrão SQL: `0`; padrão Python: `0`.
`imagem_ultima_falha`	TEXT / sim	Descrição da falha recente.
`imagem_motivo_rejeicao`	TEXT / sim	Motivo de rejeição da referência.
`criado_em`	timestamp com fuso / não	Instante de criação. padrão SQL: `now()`.
`atualizado_em`	timestamp com fuso / sim	Instante da última atualização; ORM preenche ao alterar.

Fonte: metadados ORM do projeto (2026).

2.4.2.3 farmacias

Cadastro dos estabelecimentos e coordenadas.

Quadro 7 – Dicionário: farmacias

Coluna	Tipo / nulo	Descrição e restrições
`id`	INTEGER / não	Identificador do registro. PK.
`cnpj`	VARCHAR(14) / não	CNPJ da farmácia. único.
`razao_social`	VARCHAR(150) / não	Razão social cadastrada.
`nome_fantasia`	VARCHAR(100) / não	Nome comercial da farmácia.
`telefone`	VARCHAR(20) / sim	Contato telefônico.
`endereco_completo`	VARCHAR(255) / não	Endereço cadastrado.
`latitude`	NUMERIC(9, 6) / sim	Latitude do estabelecimento.
`longitude`	NUMERIC(10, 6) / sim	Longitude do estabelecimento.
`criado_em`	timestamp com fuso / não	Instante de criação. padrão SQL: `now()`.
`atualizado_em`	timestamp com fuso / sim	Instante da última atualização; ORM preenche ao alterar.

Fonte: metadados ORM do projeto (2026).

2.4.2.4 usuario

Conta; nome é declarado localmente e os demais campos são herdados de SQLAlchemyBaseUserTableUUID.

Quadro 8 – Dicionário: usuario

Coluna	**Tipo / nulo**	**Descrição e restrições**
`nome`	VARCHAR(100) / não	Nome cadastrado do usuário.
`id`	UUID / não	Identificador do registro. PK; padrão Python: `uuid4`.
`email`	VARCHAR(320) / não	E-mail de autenticação. único.
`hashed_password`	VARCHAR(1024) / não	Resumo criptográfico da senha; nunca resposta pública.
`is_active`	BOOLEAN / não	Indicador de conta ativa. padrão Python: `True`.
`is_superuser`	BOOLEAN / não	Indicador de privilégio administrativo. padrão Python: `False`.
`is_verified`	BOOLEAN / não	Indicador de verificação da conta. padrão Python: `False`.

Fonte: metadados ORM do projeto (2026).

2.4.2.5 ofertas_farmacia

Ocorrências comerciais de produtos em farmácias.

Quadro 9 – Dicionário: ofertas_farmacia

Coluna	**Tipo / nulo**	**Descrição e restrições**
`id`	INTEGER / não	Identificador do registro. PK.
`sku_interno`	VARCHAR(50) / não	Identificador do produto na farmácia; não é EAN.
`preco`	NUMERIC(10, 2) / não	Preço observado, com duas casas decimais.
`quantidade_estoque`	INTEGER / não	Quantidade informada pela coleta, sem garantia de estoque físico. padrão Python: `0`.
`disponivel`	BOOLEAN / não	Disponibilidade observada na fonte. padrão Python: `True`.
`url_origem`	VARCHAR(500) / não	Página pública de origem da oferta. único.
`imagem_url`	VARCHAR(500) / sim	Referência externa da imagem; não armazena a fotografia.
`criado_em`	timestamp com fuso / não	Instante de criação. padrão SQL: `now()`.
`atualizado_em`	timestamp com fuso / sim	Instante da última atualização; ORM preenche ao alterar.
`observado_em`	timestamp com fuso / sim	Instante da observação da oferta.
`farmacia_id`	INTEGER / não	Vínculo com a farmácia. FK `farmacias.id`.
`catalogo_id`	INTEGER / não	Vínculo com o item do catálogo. FK `catalogo_base.id`.

Fonte: metadados ORM do projeto (2026).

2.4.2.6 usuario_farmacia_favorita

Associação de usuários com farmácias favoritas, sem coluna id própria.

Quadro 10 – Dicionário: usuario_farmacia_favorita

Coluna	**Tipo / nulo**	**Descrição e restrições**
`usuario_id`	UUID / não	Vínculo com o usuário. PK composta; FK `usuario.id`. ON DELETE CASCADE.
`farmacia_id`	INTEGER / não	Vínculo com a farmácia. PK composta; FK `farmacias.id`. ON DELETE CASCADE.

Fonte: metadados ORM do projeto (2026).

2.4.2.7 usuario_medicamento_favorito

Associação de usuários com medicamentos favoritos, sem coluna id própria.

Quadro 11 – Dicionário: usuario_medicamento_favorito

Coluna	**Tipo / nulo**	**Descrição e restrições**
`usuario_id`	UUID / não	Vínculo com o usuário. PK composta; FK `usuario.id`. ON DELETE CASCADE.
`medicamento_id`	INTEGER / não	Vínculo com o item do catálogo. PK composta; FK `catalogo_base.id`. ON DELETE CASCADE.

Fonte: metadados ORM do projeto (2026).

2.4.2.8 historico_precos

Observações de preço vinculadas a produto e farmácia.

Quadro 12 – Dicionário: historico_precos

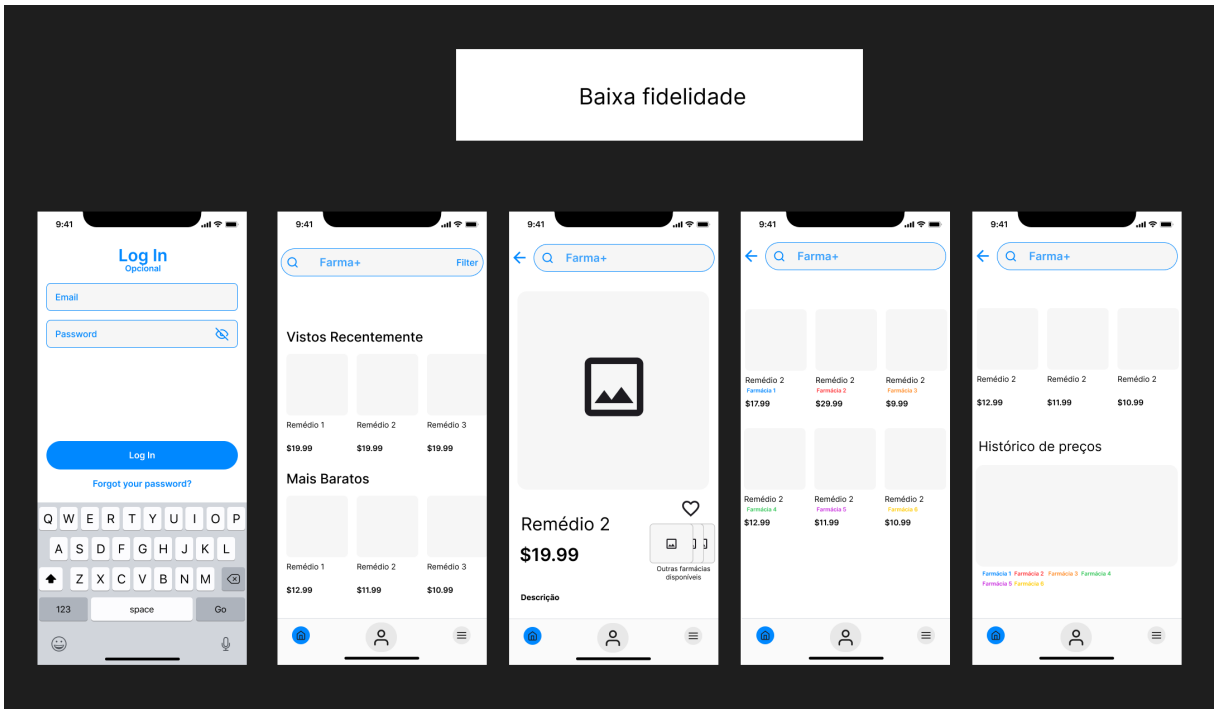
Coluna	**Tipo / nulo**	**Descrição e restrições**
`id`	INTEGER / não	Identificador do registro. PK.
`medicamento_id`	INTEGER / não	Vínculo com o item do catálogo. FK `catalogo_base.id`.
`farmacia_id`	INTEGER / não	Vínculo com a farmácia. FK `farmacias.id`.
`oferta_id`	INTEGER / sim	Oferta associada à observação; registros legados podem não ter vínculo. FK `ofertas_farmacia.id`.
`origem`	VARCHAR(40) / sim	Origem da observação de preço.
`preco`	NUMERIC(10, 2) / não	Preço observado, com duas casas decimais.
`disponivel`	BOOLEAN / sim	Disponibilidade observada na fonte.
`data_registro`	timestamp sem fuso / não	Instante do registro histórico, sem fuso no banco. padrão SQL: `now()`.

Fonte: metadados ORM do projeto (2026).

2.5 Projeto de interfaces e navegação

A imagem preservada é um protótipo de baixa fidelidade, assim identificado no próprio arquivo. Não é captura da aplicação atual. Os metadados do PNG indicam exportação pelo Figma. Não foi localizado o projeto editável nem registro de validação com usuários. A Figura 6 mantém a composição original; as Figuras 7, 8 e 9 ampliam seus rótulos.

Figura 6 – Visão geral do protótipo de baixa fidelidade

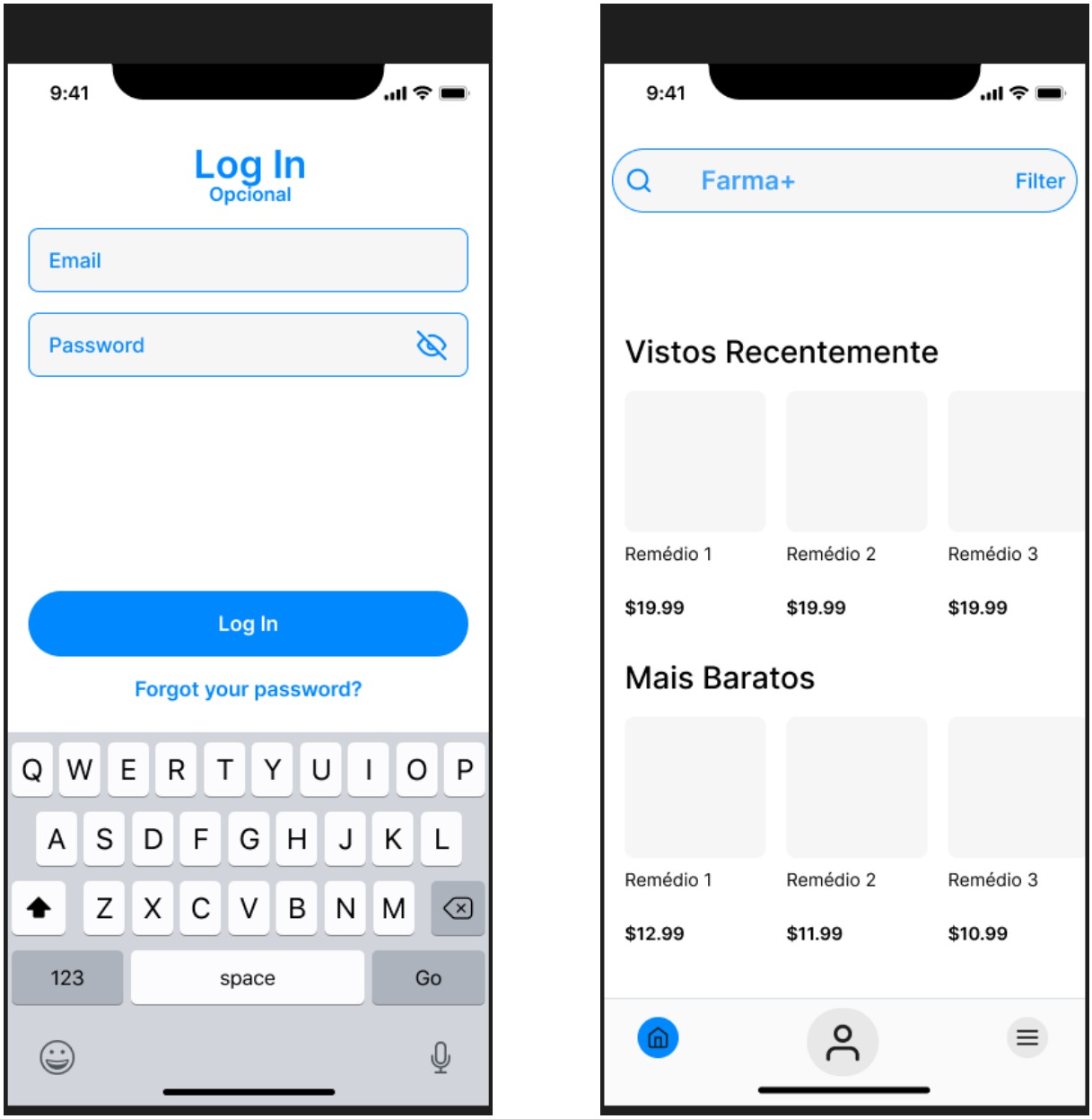


Fonte: Protótipo preservado no repositório, arquivo `prototipo-telas.png`.

A ordem real, da esquerda para a direita, é: login, início, detalhe do produto, resultados de busca e histórico. Login é opcional para consulta; favoritos e perfil requerem sessão ativa. O protótipo contém rótulos em inglês e valores ilustrativos; estes não são evidências de preços coletados.

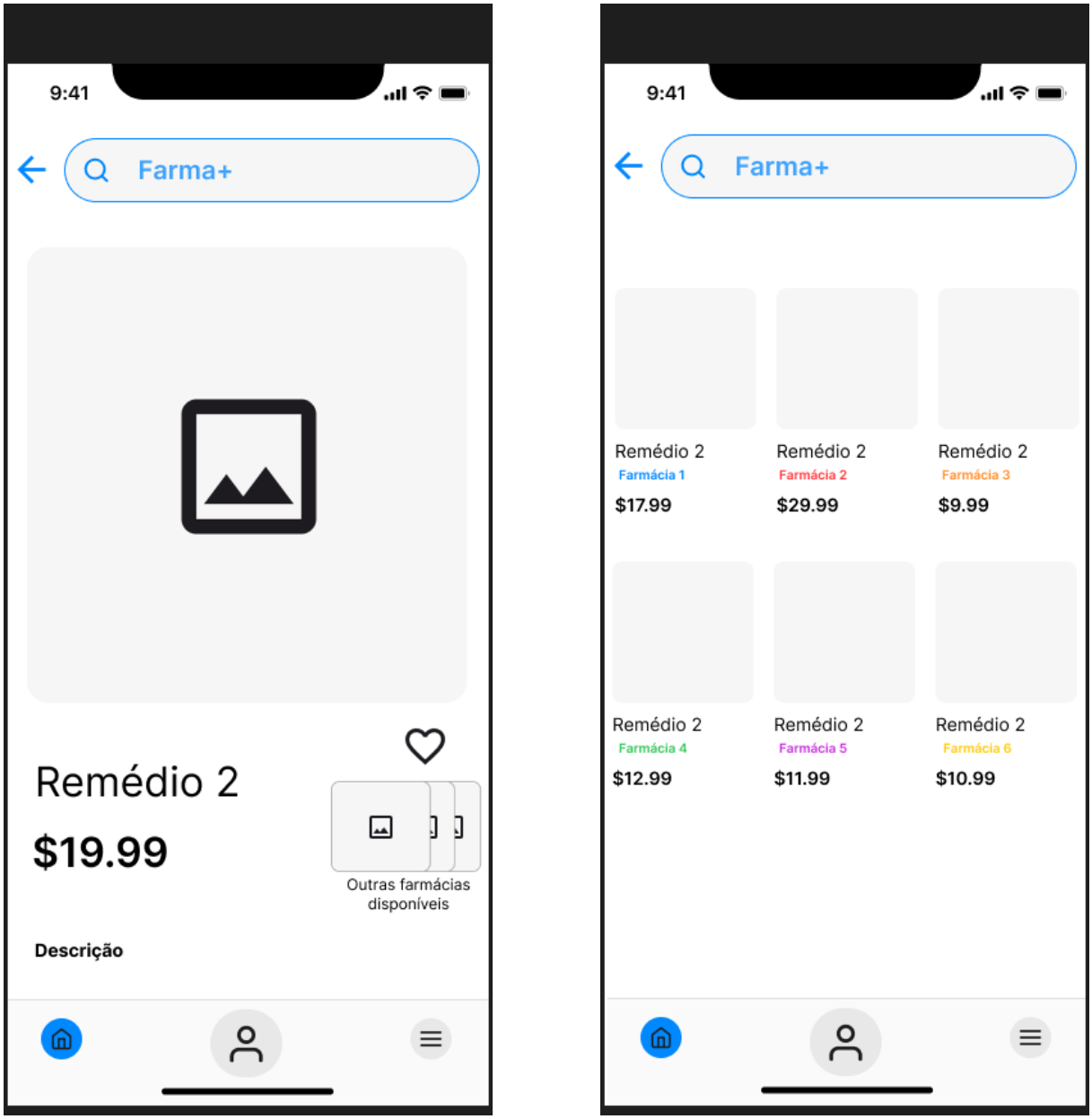
- **Conta (UC03/RF03):** login solicita e-mail e senha e mostra opção de recuperação. A opção desenhada não comprova implementação de recuperação; as telas implementadas de login e cadastro estão em `frontend/src/app/`.
- **Início e busca (UC04/RF04–RF05):** protótipo mostra pesquisa, itens e cartões. A implementação usa rotas de início e busca, filtros e `MedicineCard.tsx`. O detalhe apresenta ofertas; preço observado não garante disponibilidade física.
- **Favoritos e histórico (UC06–UC07/RF07–RF08):** o coração no detalhe indica salvamento pretendido; a persistência atual é implementada pelos hooks de favoritos e pelas rotas autenticadas. A tela de histórico corresponde à consulta de séries, atualmente composta por `HistoryChart.tsx`.

Figura 7 – Ampliação do protótipo: login e início



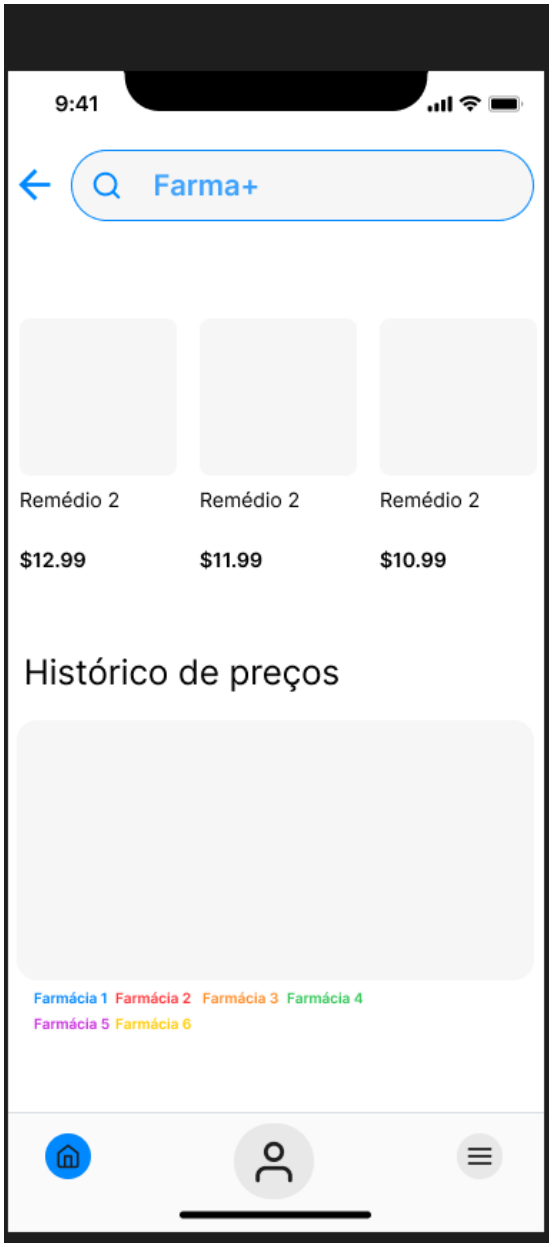
Fonte: Recortes do protótipo preservado no repositório.

Figura 8 – Ampliação do protótipo: detalhe e pesquisa



Fonte: Recortes do protótipo preservado no repositório.

Figura 9 – Ampliação do protótipo: histórico de preços



Fonte: Recorte do protótipo preservado no repositório.

- **Distância (UC05/RF06):** permanece uma jornada prevista; não há fundamento para descrever quilometragem exata na imagem ou no fluxo implementado.

Não há ensaio de contraste, tamanho de alvos de toque ou navegação assistiva que sustente conformidade WCAG. Hierarquia visual e ícones são decisões de projeto; acessibilidade efetiva requer avaliação específica.

2.6 Plano de Testes

Este tópico apresenta o plano de testes da plataforma Preço Bão, mantendo alinhamento com a especificação de requisitos, modelagem, arquitetura e protótipo de interface documentados no projeto. O objetivo é planejar a verificação e validação das funcionalidades principais do sistema, incluindo pesquisa de medicamentos, comparação de preços, geolocalização, autenticação, favoritos, monitoramento de preços e rotina automatizada de coleta de dados por *web scraping*.

2.6.1 Introdução

- **Nome do sistema:** Plataforma Preço Bão.
- **Objetivo do software:** Permitir que consumidores pesquisem, comparem e monitorem preços de medicamentos e produtos de farmácia por meio de aplicativo móvel e versão web.
- **Problema que o sistema busca resolver:** A dificuldade do consumidor em comparar preços de medicamentos em diferentes farmácias, considerando volatilidade dos valores, disponibilidade estimada em estoque e distância até os estabelecimentos.
- **Público-alvo:** Consumidores finais que compram medicamentos de uso contínuo, produtos de farmácia, higiene e dermocosméticos, especialmente usuários que desejam economizar tempo e dinheiro.
- **Integrantes do grupo:** Aurora Drumond Costa Magalhães, Ana Clara de Souza e Kayke Wellington Ribeiro Santos.

2.6.2 Escopo dos Testes

O escopo dos testes contempla as funcionalidades centrais necessárias para validar o funcionamento do sistema sob a perspectiva do usuário final, do banco de dados e dos serviços de retaguarda.

Funcionalidades que serão testadas:

- Cadastro, *login*, autenticação e recuperação de senha de usuários.
- Pesquisa de medicamentos e produtos de farmácia.
- Exibição dos resultados em cartões padronizados, contendo laboratório, preço, farmácia, disponibilidade estimada e origem da oferta.
- Ordenação neutra dos resultados por menor preço ou menor distância.
- Cálculo de distância geográfica mediante autorização de localização.
- Favoritar medicamentos e produtos para acompanhamento posterior.
- Exibição de histórico ou gráfico de variação de preços.
- Rotina automatizada de *web scraping*, tratamento de falhas e persistência dos dados coletados.
- Integração entre *frontend* React Native, *backend* FastAPI e banco PostgreSQL.

Funcionalidades fora do escopo:

- Compra, pagamento ou reserva de medicamentos dentro da plataforma.
- Confirmação oficial de estoque em tempo real diretamente no sistema da farmácia.
- Entrega em domicílio ou integração com sistemas logísticos.
- Validação regulatória completa de prescrição médica ou controle de receita.

Limitações identificadas:

- A disponibilidade em estoque é uma estimativa e deve ser confirmada no local pelo usuário.
- A precisão dos dados depende da qualidade das páginas públicas das farmácias e da rotina de extração.
- A geolocalização depende da permissão do usuário e da precisão do dispositivo.
- Testes de carga em ambiente acadêmico podem não representar a escala real de produção.

2.6.3 Requisitos a Serem Testados

Os requisitos selecionados foram extraídos da especificação do projeto e representam os comportamentos funcionais e critérios de qualidade mais relevantes para a validação da plataforma.

Quadro 13 – Requisitos funcionais selecionados para teste

ID	Requisito a ser testado
RF01	**Extração Resiliente de Dados:** validar se a rotina de *web scraping* coleta dados de farmácias, registra falhas em *log* e continua a varredura nas demais fontes.
RF02	**Processamento e Catálogo:** verificar persistência, enriquecimento CMED/ANVISA e aviso de estoque estimado, incluindo ausência de correspondência.
RF03	**Gestão de Usuários:** verificar cadastro, autenticação, *login* e recuperação de senha do usuário.
RF04	**Pesquisa e Neutralidade Algorítmica:** validar busca por medicamentos e ordenação por menor preço ou menor distância.
RF05	**Exibição em Cartões:** verificar se os resultados são exibidos com informações de laboratório, preço e melhores ofertas.
RF06	**Cálculo de Distância Geográfica:** verificar cálculo de distância até as farmácias após permissão de localização.
RF07	**Salvamento de Produtos:** validar a opção de favoritar medicamentos de uso contínuo na conta do usuário.
RF08	**Monitoramento de Preços:** verificar o histórico ou gráfico de flutuação de preços dos itens salvos.

Fonte: Elaborado pelos autores (2026).

Quadro 14 – Requisitos não funcionais selecionados para teste

ID	Requisito a ser testado
RNF01	**Performance da API:** validar resposta de requisições de pesquisa em até 500 milissegundos no percentil 95.
RNF02	**Arquitetura do Frontend:** verificar execução do aplicativo React Native/Expo com TypeScript para Android e iOS.
RNF04	**Segurança e Criptografia:** validar armazenamento de senhas com *hash* forte e comunicação protegida por HTTPS/TLS.
RNF05	**Privacidade (LGPD):** verificar se coordenadas geográficas são processadas em memória e descartadas após uso.
RNF08	**Orquestração Local:** validar provisionamento do ambiente com `docker-compose.yml`.
RNF10	**CI/CD e Qualidade:** verificar bloqueio de *Pull Requests* que falhem em testes ou análise estática.

Fonte: Elaborado pelos autores (2026).

2.6.4 Estratégia de Testes

A estratégia de testes combina validação funcional, integração, persistência, segurança, interface e desempenho, cobrindo tanto o fluxo do usuário quanto os componentes internos

da arquitetura.

Quadro 15 – Estratégias de teste propostas

Tipo de Teste	**Objetivo**	**Técnica Utilizada**	**Resultado Esperado**
Teste Funcional	Verificar se as funcionalidades do sistema executam corretamente conforme os requisitos.	Executar fluxos com dados válidos e inválidos em cadastro, *login*, pesquisa, favoritos e histórico.	O sistema deve responder corretamente às ações do usuário e exibir mensagens adequadas.
Teste de Interface	Avaliar usabilidade, organização visual e navegação entre as telas.	Navegar pelo aplicativo validando botões, campos, cartões, barra inferior, contraste e rótulos.	As telas devem permitir concluir as jornadas avaliadas, com rótulos e ações legíveis, registrando dificuldades observadas.
Teste de Banco de Dados	Verificar persistência, consulta e integridade das informações.	Inserir, consultar e validar registros em Usuario, CatalogoBase, Farmacia, OfertaFarmacia, associações de favoritos e HistoricoPreco.	Os dados devem ser armazenados e recuperados sem inconsistências.
Teste de Segurança	Garantir proteção de credenciais e acesso restrito a recursos autenticados.	Testar *login* inválido, acesso a favoritos sem autenticação, *hash* de senha e descarte de localização.	Usuários não autorizados não devem acessar recursos restritos, e dados sensíveis não devem ser expostos.
Teste de Integração	Verificar comunicação entre *frontend*, API, banco e rotina de *scraping*.	Executar fluxo completo: coleta de dados, gravação no PostgreSQL, consulta via FastAPI e exibição no app.	Os módulos devem trocar informações corretamente e tratar falhas sem interromper o sistema.
Teste de Desempenho	Avaliar comportamento da API em pesquisas simultâneas.	Simular múltiplas requisições e medir tempo de resposta das buscas principais.	A API deve manter tempo de resposta compatível com RNF01.

Fonte: Elaborado pelos autores (2026).

2.6.5 Casos de Teste

Os casos de teste foram definidos para cobrir os fluxos principais da plataforma. Todos permanecem **Pendentes**: os resultados descritos são esperados, não observados. CT13 foi detalhado para incluir RF02 e CT14 para explicitar o critério de RNF01; esses detalhamentos pertencem à consolidação documental, sem execução histórica atribuída à equipe.

A rastreabilidade funcional é: RF01–CT12; RF02–CT13; RF03–CT01/CT02/CT03 (recuperação de senha ainda sem caso específico); RF04–CT04/CT05/CT06/CT07; RF05–CT04; RF06–CT07/CT08; RF07–CT09/CT10; RF08–CT11. A cobertura de RNF01 está em CT14. Hash, HTTPS, descarte de localização, orquestração e bloqueio de PRs aparecem nas estratégias, mas carecem de procedimentos e registros específicos. Assim, a seleção de requisitos não equivale a cobertura integral validada.

Quadro 16 – Casos de teste da plataforma Preço Bão - Prioridade Alta

ID	**Caso de Teste**	**Entrada de Dados**	**Resultado Esperado**
CT01	Cadastro de usuário com dados válidos	Nome, e-mail válido e senha válida	Usuário cadastrado com sucesso e perfil criado.
CT02	Login com credenciais válidas	E-mail e senha corretos	Acesso permitido à conta do usuário.
CT03	Login com senha inválida	E-mail válido e senha incorreta	Sistema deve negar acesso e exibir mensagem de credenciais inválidas.
CT04	Pesquisa de medicamento existente	Nome de medicamento presente no catálogo	Sistema deve listar ofertas disponíveis em cartões padronizados.
CT06	Ordenação por menor preço	Busca com múltiplas ofertas	Resultados devem ser ordenados pelo menor preço, sem favorecimento de farmácia.
CT07	Ordenação por menor distância	Busca com permissão de localização ativa	Resultados devem ser ordenados pela menor distância calculada até as farmácias.
CT09	Favoritar medicamento autenticado	Usuário logado seleciona favorito	Produto deve ser associado ao usuário e aparecer na tela de favoritos.
CT10	Favoritar medicamento sem login	Usuário não autenticado tenta favoritar	Sistema deve solicitar login antes de salvar o produto.
CT12	Scraping com falha parcial	Uma fonte indisponível durante a coleta	Erro deve ser registrado em log e a varredura deve continuar nas demais farmácias.

Continua na próxima página.

Quadro 16 – continuação

ID	Caso de Teste	Entrada de Dados	Resultado Esperado
CT13	Persistência de oferta coletada	Produto, farmácia, preço e disponibilidade coletados	Dados devem ser gravados em OfertaFarmacia e vinculados ao catálogo e à farmácia; conferir enriquecimento CMED/ANVISA, ausência de correspondência e aviso de estoque estimado (RF02).

Fonte: Elaborado pelos autores (2026).

Quadro 17 – Casos de teste da plataforma Preço Bão - Prioridade Média

ID	Caso de Teste	Entrada de Dados	Resultado Esperado
CT05	Pesquisa sem resultado	Nome inexistente ou inválido	Sistema deve informar que nenhum produto foi encontrado.
CT08	Negar permissão de localização	Usuário recusa acesso à localização	Sistema deve manter pesquisa por preço e informar indisponibilidade da ordenação por distância.
CT11	Exibir histórico de preços	Produto salvo com registros em HistoricoPreco	Sistema deve apresentar gráfico ou lista de variação de preços das últimas semanas.

Fonte: Elaborado pelos autores (2026).

Quadro 18 – Casos de teste da plataforma Preço Bão - Prioridade Baixa

ID	Caso de Teste	Entrada de Dados	Resultado Esperado
CT14	Teste de desempenho da busca	Múltiplas requisições simultâneas de pesquisa	O percentil 95 dos tempos de resposta deve ser menor ou igual a 500 ms (RNF01); registrar carga, ambiente, duração, amostra e método de cálculo.

Fonte: Elaborado pelos autores (2026).

2.6.6 Infraestrutura e Recursos de Teste

- **Sistema operacional:** Windows 10/11 ou Linux para execução dos serviços de desenvolvimento e testes.
- **Banco de dados:** PostgreSQL, conforme especificação de persistência do projeto.

- **Backend:** Python com FastAPI, SQLAlchemy ORM, Pydantic e execução assíncrona.
- **Frontend:** React Native com Expo e TypeScript.
- **Navegador:** Google Chrome, Microsoft Edge ou navegador compatível para validação da versão web e consumo da API.
- **Ferramentas:** Visual Studio Code, Docker, docker-compose, Git/GitHub, GitHub Actions, ESLint, Ruff e ferramentas de teste da API.
- **Equipamentos:** Computador de desenvolvimento e smartphone ou emulador Android/iOS para validação do aplicativo móvel.

2.6.7 Equipe e Responsabilidades

O quadro preserva a distribuição de responsabilidades registrada no plano. As funções são atribuições planejadas; não comprovam execução, participação ou resultados individuais.

Quadro 19 – Equipe e responsabilidades

Integrante	**Função**
Aurora Drumond Costa Magalhães	Elaboração dos casos de teste e validação dos requisitos funcionais.
Ana Clara de Souza	Execução dos testes de interface, autenticação, favoritos e registro das falhas encontradas.
Kayke Wellington Ribeiro Santos	Execução dos testes de *backend*, banco de dados, *scraping*, desempenho e integração.
Grupo	Revisão final do plano de testes, consolidação dos resultados e validação do documento.

Fonte: Elaborado pelos autores (2026).

2.6.8 Cronograma de Testes

O cronograma a seguir preserva as datas e atribuições do plano. A ordem das linhas não é estritamente cronológica: as etapas de 13 e 14 de junho antecedem a execução funcional e de interface de 16 de junho. Sem registros de execução, não se presume que as datas estejam erradas nem que o registro de falhas de 14 de junho abranja a etapa posterior.

Quadro 20 – Cronograma simplificado de testes

Atividade	**Responsável**	**Prazo**
Planejamento dos testes	Grupo	03/06/2026

Continua na próxima página.

Quadro 20 – continuação

Atividade	**Responsável**	**Prazo**
Elaboração dos casos de teste	Aurora Drumond Costa Magalhães	08/06/2026
Preparação do ambiente de teste	Kayke Wellington Ribeiro Santos	12/06/2026
Execução dos testes funcionais e de interface	Ana Clara de Souza	16/06/2026
Execução dos testes de banco, integração, segurança e desempenho	Kayke Wellington Ribeiro Santos	13/06/2026
Registro de falhas e análise dos riscos	Grupo	14/06/2026
Correção, validação e revisão final	Grupo	17/06/2026

Fonte: Elaborado pelos autores (2026).

2.6.9 Registro de Defeitos e Riscos

O registro de defeitos e riscos é prospectivo e não relata falhas reproduzidas. Ele antecipa problemas que podem ocorrer durante a execução dos testes e durante a evolução do projeto.

Quadro 21 – Hipóteses de defeitos para investigação

ID	**Hipótese de falha**	**Possível Causa**	**Solução Proposta**
RFAL01	Login não permite acesso mesmo com credenciais corretas.	Erro na validação da senha, *hash* incorreto ou falha de comunicação com o banco.	Revisar autenticação, geração de *hash* e consulta do usuário no *backend*.
RFAL02	Pesquisa retorna produtos duplicados ou sem preço.	Falha no tratamento dos dados coletados pelo *scraping* ou inconsistência em OfertaFarmacia.	Revisar normalização do catálogo, chaves de relacionamento e filtros da consulta.
RFAL03	Distância geográfica exibida incorretamente.	Erro no cálculo de latitude/longitude, permissão negada ou coordenadas inválidas.	Validar fórmula de cálculo, tratar ausência de localização e testar valores conhecidos.
RFAL04	Histórico de preços não aparece para produto favoritado.	Ausência de registros em HistoricoPreco ou erro de vínculo com usuário/produto.	Revisar rotina de gravação histórica e relacionamento entre as associações de favoritos e CatalogoBase.

Fonte: Elaborado pelos autores (2026).

Quadro 22 – Riscos relacionados ao projeto e aos testes

ID	**Risco identificado**	**Impacto**	**Mitigação**
R01	Mudança no layout ou na estrutura dos sites das farmácias.	Pode interromper ou reduzir a qualidade da extração de dados por *scraping*.	Criar *logs* detalhados, testes automatizados para os *scrapers* e estratégia de adaptação por farmácia.

Continua na próxima página.

Quadro 22 – continuação

ID	Risco identificado	Impacto	Mitigação
R02	Ambiente de testes diferente do ambiente real de produção.	Resultados de desempenho e integração podem não representar a operação real.	Documentar limitações, executar testes em ambiente controlado e repetir cenários críticos em infraestrutura próxima da final.
R03	Exposição indevida de dados pessoais ou localização.	Pode causar não conformidade com a LGPD e perda de confiança do usuário.	Validar descarte das coordenadas, revisar permissões e testar fluxos de privacidade.

Fonte: Elaborado pelos autores (2026).

3 Configuração e Manutenção de Software

Este capítulo reúne os produtos documentais das Práticas 1 e 2 da disciplina Configuração e Manutenção de Software. As fichas identificam o oitavo período e a professora Maíza Cristina de Souza Dias. A Prática 1 apresenta alternativas para reduzir dificuldades de manutenção; a Prática 2 organiza demandas, classificação, prioridades e plano de ação. As respostas registram análises e propostas do projeto “Preço Bão”, sem constituir, por si, comprovação de execução das alterações propostas.

Os registros de referência são a ficha preenchida da Prática 1, em formato Word, e a ficha preenchida da Prática 2, em PDF, preservadas com esta documentação (ATIVIDADE PRÁTICA 01. . . , [s.d.]; ATIVIDADE PRÁTICA 02. . . , [s.d.]). Os campos de data estão em branco nos dois materiais; não se atribuem datas de realização a partir de nomes de arquivos ou de sua ordem numérica. As propostas são relacionadas à implementação descrita no Capítulo 2, mantendo a distinção entre o planejamento registrado e o código consultado.

3.1 Prática 1: alternativas para a manutenção do sistema

3.1.1 Solicitação e produto elaborado

A atividade solicita a leitura e a síntese do artigo *Alternativas para redução de problemas na manutenção de software*, a retomada dos componentes e desafios do sistema, a identificação de problemas potenciais e a proposição de soluções, considerando custo, prazo e impacto. Também solicita documentar e apresentar as propostas e disponibilizar a documentação no ambiente virtual de aprendizagem (ATIVIDADE PRÁTICA 01. . . , [s.d.]).

O produto recebido contém respostas a esses tópicos, uma síntese para apresentação e quatro itens de manutenção classificados por tipo e gravidade. Sua contribuição é a análise das alternativas e a organização de uma proposta de intervenção. As estimativas apresentadas são previsões da ficha, sem medição de esforço realizado. A existência desse documento não comprova apresentação em aula, postagem no ambiente virtual ou aplicação das soluções.

3.1.2 Síntese da leitura e alternativas selecionadas

A resposta da ficha destaca treinamento, base de conhecimento, documentação, formalização de chamados, acompanhamento de cronogramas, divisão de tarefas e melhoria dos testes. O artigo indicado relaciona alternativas de manutenção à orientação da ISO/IEC

12207, às observações de uma organização e à literatura da área. Nele, a melhoria de documentação interna ainda aparece como dificuldade sem solução satisfatória, enquanto a base de conhecimento e o fluxo de informações são práticas descritas. Assim, atualização documental integra a proposta da equipe, sem se afirmar que todos esses mecanismos já estavam resolvidos no estudo apresentado por Mateus (2011).

Aplicadas ao Preço Bão como planejamento, essas alternativas compreendem capacitação em FastAPI, PostgreSQL, Docker, Expo e coleta de dados; registro de problemas e soluções; atualização de instruções de instalação e contratos da API; uso de chamados com prioridade e critérios de aceite; revisão de código; testes antes da liberação; e acompanhamento da disponibilidade e da qualidade dos dados. Essas ações procuram reduzir dependência de conhecimento individual, divergências entre ambientes e regressões nas funções centrais.

3.1.3 Componentes e dificuldades retomados

A ficha descreve o frontend em React Native, Expo, Expo Router e TypeScript; o backend Python/FastAPI; a persistência PostgreSQL com SQLAlchemy e Alembic; os coletores de farmácias; e o ambiente Docker Compose, com configurações em arquivos de ambiente. Relaciona essas partes à pesquisa, aos filtros, à comparação de ofertas, à conta, aos favoritos e ao histórico (ATIVIDADE PRÁTICA 01. . . , [s.d.]). Os artefatos técnicos correspondentes estão nas Seções 2.2.1, 2.4 e 2.5.

Os desafios relatados envolvem integração entre frontend e backend, URL da API e CORS, preparação de máquinas distintas, migrações e carga inicial, persistência dos favoritos, adaptação da interface, dependências Android e alimentação do catálogo pelos coletores. O registro menciona interface web e Android, mas não apresenta resultado de teste em dispositivo; essa menção não altera a distinção entre componente presente e funcionamento validado.

3.1.4 Problemas potenciais e propostas de manutenção

A análise identifica riscos de mudanças nos sites de farmácias, falhas de coleta, preços desatualizados, duplicação e incompletude do catálogo; indisponibilidade da API ou do banco; erros de ambiente; perda de volumes e ausência de backup; incompatibilidade de dependências; desalinhamento de contratos de autenticação e favoritos; crescimento do volume de consultas; e dificuldade de suporte sem registros padronizados.

O Quadro 23 preserva os quatro itens sintetizados na entrega. Seus números identificam os itens da Prática 1; não são equivalências automáticas dos identificadores M-01–M-04 da prática seguinte.

Quadro 23 – Itens de manutenção propostos na Prática 1

Item	**Ocorrência e efeito**	**Tipo**	**Prioridade / gravidade**
01	Mudança no site de uma farmácia interrompe a coleta e a atualização de preços, comprometendo a comparação de ofertas.	Adaptativa e corretiva	Alta / crítica.
02	Mudança ou expiração do endereço do túnel, CORS ou variáveis de ambiente interrompe a comunicação com a API.	Adaptativa e preventiva	Alta.
03	Banco vazio, migrações ou carga inicial pendentes e falta de backup comprometem catálogo, contas e histórico.	Corretiva e preventiva	Alta / crítica.
04	Atualizações de dependências podem impedir compilação, instalação ou execução em novos ambientes.	Adaptativa e preventiva	Média / alta.

Fonte: ficha preenchida da Prática 1.

As soluções propostas incluem testes unitários e de integração para conta, catálogo, filtros, favoritos e coletores; homologação; monitoramento e registros de falha; verificações de disponibilidade dos serviços; backup e validação dos dados; automação das migrações; e substituição de endereço temporário de túnel por configuração estável. O Quadro 24 mantém as estimativas e os efeitos esperados declarados na ficha, sem convertê-los em resultados obtidos.

Quadro 24 – Estimativas propostas na Prática 1

Ação	**Custo estimado**	**Prazo estimado**	**Efeito esperado**
Documentação e base de conhecimento	Baixo	2 a 4 dias	Reduzir tempo de suporte e dependência de conhecimento individual.
Testes automatizados	Médio	1 a 2 semanas	Reduzir regressões nos fluxos principais.
Monitoramento e registros	Médio	3 a 5 dias	Detectar indisponibilidade de API, banco e coletores.

Continua na próxima página.

Quadro 24 – continuação

Ação	**Custo estimado**	**Prazo estimado**	**Efeito esperado**
Banco, migrações e backup	Baixo a médio	2 a 4 dias	Reduzir risco de catálogo vazio e perda de dados.
Refatoração dos coletores	Médio a alto	1 a 3 semanas	Melhorar adaptação a alterações das fontes.
Túnel ou domínio estável	Baixo a médio	1 a 2 dias	Reduzir interrupções por mudança de URL.
CI/CD e revisão de código	Médio	3 a 7 dias	Propor testes e análise estática antes da integração.

Fonte: ficha preenchida da Prática 1.

3.1.5 Síntese da proposta

A entrega prioriza estabilidade da coleta, preparação e proteção do banco e disponibilidade da API; em seguida, propõe testes, monitoramento, documentação e padronização de chamados. Não há valores monetários, medições de redução de falhas ou comprovação de execução no material recebido. A meta de CI/CD permanece sujeita à distinção de implementação registrada na Seção 2.2.1.

3.2 Prática 2: identificação e planejamento de manutenções

3.2.1 Solicitação, classificação e produto elaborado

A atividade solicita identificar e documentar problemas após a implantação, classificá-los, priorizar um backlog inspirado em Scrum, detalhar um plano de ação, organizar sprints e refletir sobre criticidade e métricas. A ficha preenchida apresenta quatro demandas M-01–M-04, sua classificação, backlog, plano coletivo para M-01, duas sprints planejadas e respostas à reflexão final (ATIVIDADE PRÁTICA 02. . . , [s.d.]).

A classificação adotada no roteiro distingue manutenção corretiva para correção de falhas, adaptativa para adequação a mudanças externas, evolutiva ou perfectiva para novos recursos e melhorias e preventiva para redução de falhas latentes e dificuldades de manutenção. O enunciado menciona ISO/IEC 14764 e IEEE 1219 sem especificar edições; a terminologia aqui reproduz a classificação da atividade, sem declarar conformidade normativa do processo do projeto.

O plano de M-01 é apresentado como produto coletivo, sem atribuição de autoria individual aos integrantes. As funções de frontend, backend, mobile e DevOps nos quadros são papéis planejados da entrega e não registros de participação ou de execução de pessoas específicas.

3.2.2 Etapas 1 e 2: levantamento, classificação e análise

O Quadro 25 consolida as ocorrências descritas e sua classificação. Solicitantes são os papéis registrados na ficha; não se presume a existência de entrevista, chamado ou incidente reproduzido para cada linha.

Quadro 25 – Demandas e classificação de manutenção da Prática 2

ID	Solicitante, ocorrência e área	Impacto e tipo	Justificativa e risco
M-01	Usuário final: favoritos podem não reaparecer após sair e autenticar novamente, apesar da persistência no backend. Funcionalidade, persistência e usabilidade.	Alto; corretiva.	Corrigir a reconstrução de uma função existente. A divergência entre estado local e servidor pode parecer perda de dados e gerar repetição de operações.
M-02	Equipe técnica/mobile: atualizações de Expo, React Native, SDK/NDK Android, bibliotecas web e navegadores podem exigir ajustes. Compatibilidade, plataforma e infraestrutura.	Médio/alto; adaptativa.	Preservar funcionamento diante de mudança externa, sem alterar regras de negócio. Incompatibilidades podem impedir compilação, instalação ou comunicação com a API.
M-03	Cliente/Product Owner: adicionar histórico visual e alertas de redução de preços. Funcionalidade e experiência do usuário.	Médio; evolutiva ou perfectiva.	Agregar recursos ao acompanhamento de medicamentos. A falta da evolução limita o valor percebido, sem implicar interrupção imediata do sistema.
M-04	Equipe técnica/DevOps: padronizar ambiente, migrações, banco, carga inicial, verificações de disponibilidade e testes. Manutenibilidade, dados, DevOps e confiabilidade.	Alto; preventiva.	Reduzir falhas de implantação e regressões. Configuração incompleta, banco vazio ou tabelas desatualizadas podem impedir a exibição de medicamentos e elevar o suporte.

Fonte: ficha preenchida da Prática 2, etapas 1 e 2.

A classificação mantém o sentido registrado na atividade, sem apagar propostas por já existirem componentes relacionados no código. Em particular, M-03 reúne histórico e alertas: a implementação consultada já contém consulta e gráfico de histórico, conforme a Seção 2.2.1; isso não demonstra a realização da sprint ou a entrega de alertas. M-01 também permanece como análise da ficha, sem converter a presença dos hooks de favoritos em prova de execução daquele plano.

3.2.3 Etapa 3: priorização e backlog

Os critérios registrados são impacto no usuário, risco de continuidade, custo de correção e urgência. O Quadro 26 conserva a ordem de priorização, as sprints e os responsáveis por papel. A criticidade de M-04 para continuidade, discutida na reflexão, não altera a ordem M-01, M-04, M-02 e M-03 apresentada no backlog.

Quadro 26 – Backlog de manutenção proposto na Prática 2

ID	Item e tipo	Prioridade	Sprint	Papel responsável
M-01	Corrigir sincronização de favoritos após novo login; corretiva.	Alta	1	Frontend e backend.
M-04	Automatizar preparação do ambiente, migrações, carga e verificações; preventiva.	Alta	1	Backend/DevOps.
M-02	Adequar frontend a mudanças de plataforma e dependências; adaptativa.	Média	2	Frontend/mobile.
M-03	Histórico de preços e alertas de redução; evolutiva.	Média	2	Frontend e backend.

Fonte: ficha preenchida da Prática 2, etapa 3.

3.2.4 Etapa 4: plano coletivo de ação para M-01

Problema e público afetado. A análise aborda a situação em que o usuário autentica novamente e a interface não reconstrói os medicamentos já persistidos como favoritos. O público afetado são contas autenticadas que utilizam esse recurso. A ficha formula o diagnóstico como causa provável, sem fornecer rastros de execução que permitam afirmá-lo como causa comprovada.

Causa provável e impacto. O estado `favoriteIds` mantido no frontend precisa ser sincronizado com a listagem `/users/me/favoritos/medicamentos`. Contrato de resposta sem o identificador esperado, atualização apenas local ou sincronização em momento inadequado podem deixar a interface divergente do banco. O efeito esperado é percepção de perda de favoritos, operações repetidas, respostas de erro em inclusões duplicadas e aumento de solicitações de suporte.

Componentes relacionados. A entrega indica `frontend/src/context/AppContext.tsx`, `frontend/src/components/MedicineCard.tsx`, `frontend/src/services/api.ts`, rotas de usuários e favoritos do FastAPI, schemas de resposta e associações de favoritos no PostgreSQL. Na estrutura atual, o contexto utiliza também `useAuthSession.ts` e `useFavoritesState.ts`; as tabelas correspondentes estão no dicionário da Seção 2.4.2. Essa relação localiza a proposta na arquitetura e não substitui seu registro por uma execução retrospectiva.

Solução proposta. A API deve retornar os identificadores dos produtos; a interface deve buscar os favoritos após autenticação e após inclusão ou remoção, usando o backend como referência persistente da conta. O plano também prevê tratar repetição de inclusão sem interromper a interface. As respostas HTTP 400/404 citadas na ficha pertencem ao cenário proposto; o comportamento atual das rotas deve ser conferido nos testes, sem alterar o registro original.

A estratégia de implementação registrada compreende:

1. revisar o contrato da listagem e garantir o campo `id`;
2. revisar as operações POST, DELETE e GET de favoritos;
3. centralizar a função de sincronização no fluxo de contexto;
4. dispará-la após login e atualizar o estado após favoritar ou desfavoritar;
5. tratar os erros previstos de forma controlada;
6. validar o fluxo na web e em Android.

O plano de testes propõe integração dos endpoints, autenticação de conta com favoritos existentes, ciclo de favoritar/sair/entrar, remoção seguida de nova autenticação, regressão da Home e da tela de favoritos, navegador e Android físico e conferência das associações no PostgreSQL. Esses cenários se relacionam a RF03 e RF07 e a CT02, CT09 e CT10 do plano geral. Os ciclos de nova autenticação e remoção detalham cenários adicionais, não cobertos integralmente pelos casos gerais citados. A ausência de resultados na ficha mantém esses ensaios como planejados; não altera os estados dos casos históricos.

Os recursos previstos são equipes de frontend e backend, Expo/React Native, FastAPI, PostgreSQL, Docker, Swagger, DevTools e Git/GitHub. O prazo de 1 a 2 dias corresponde à estimativa da atividade para correção, testes e documentação, sem identificação de datas efetivas.

3.2.5 Etapa 5: cronograma de sprints

O Quadro 27 preserva as duas sprints de uma semana e o estado **Planejado**. A ficha não informa início ou término no calendário.

Quadro 27 – Sprints planejadas na Prática 2

Sprint	**Duração, objetivo e estado**	**Itens**	**Papéis responsáveis**
1	Uma semana: corrigir falhas críticas e reforçar confiabilidade de dados e implantação. Planejado.	M-01: sincronização de favoritos. M-04: ambiente, migrações, carga e verificações.	Frontend, backend e DevOps.
2	Uma semana: manter compatibilidade e evoluir a experiência do usuário. Planejado.	M-02: plataforma e dependências. M-03: histórico e alertas.	Frontend e backend.

Fonte: ficha preenchida da Prática 2, etapa 5.

A divisão por frentes da ficha compreende equipe A, frontend/UX e integração; equipe B, backend/dados, autenticação, catálogo e coletores; e equipe C, infraestrutura/qualidade, Docker, túnel, migrações, testes e documentação. Essas denominações organizam responsabilidades propostas e não criam novos integrantes nem atribuem realizações individuais.

3.2.6 Etapa 6: reflexão e indicadores propostos

A reflexão identifica M-04 como manutenção mais crítica para a continuidade do produto, pois catálogo e banco sustentam suas consultas. A indisponibilidade dos dados pode impedir a exibição de medicamentos mesmo quando há acesso à API. A ficha menciona backend permanente no Cloudflare, enquanto a Prática 1 discute Quick Tunnel temporário e sua substituição; sem datas e configuração de implantação correspondente, não se estabelece uma cronologia de mudança nem garantia de publicação permanente. A configuração consultada permanece descrita na Seção 2.2.1.

Quanto à evolução, a entrega relaciona correção de falhas à confiabilidade, adaptação à compatibilidade, evolução ao valor para o usuário e prevenção à redução de regressões e dificuldades futuras. Como indicadores possíveis, sugere disponibilidade e tempo de resposta da API, erros HTTP 5xx, sucesso de login e cadastro, sincronização de favoritos, quantidade de medicamentos e ofertas válidas, idade dos preços, sucesso dos coletores, cobertura e aprovação de testes, defeitos reabertos e tempo médio de recuperação. Não há valores ou comparação antes/depois no registro; são propostas de avaliação, não resultados medidos.

Para gerir o backlog, a ficha propõe registrar tipo, impacto, risco, urgência e critérios de aceite; priorizar continuidade e funções centrais no planejamento da sprint; acompanhar andamento em reuniões diárias; incluir testes e documentação na definição de conclusão; validar resultados na revisão da sprint; e usar a retrospectiva para ações preventivas. O registro desse processo proposto não comprova a realização das cerimônias.

4 Conclusão

A documentação relaciona o domínio de comparação de preços farmacêuticos aos requisitos, casos de uso, classes, esquema relacional, interfaces e planejamento de testes do "Preço Bão". A implementação consultada contém componentes de catálogo, autenticação, favoritos e observações históricas de preços, organizados em backend FastAPI, frontend React Native/Expo e persistência PostgreSQL. O DER e o dicionário tornam explícitas as tabelas associativas e os atributos herdados da autenticação.

A conferência também evidencia diferenças entre metas e implementação. Recuperação de senha no fluxo público, cálculo de distância na pesquisa, retenção automática de históricos por 12 meses e bloqueio de alterações por verificações de qualidade não foram demonstrados pelas fontes consultadas. As metas de desempenho, funcionamento em dispositivos móveis e conformidade de privacidade exigem validações específicas; a presença de código ou configuração não comprova esses resultados.

As Práticas 1 e 2 acrescentam uma análise de riscos de manutenção e um planejamento de intervenções. A primeira relaciona problemas de coleta, ambiente, banco e dependências a alternativas e estimativas. A segunda classifica quatro demandas, organiza prioridades e sprints e detalha coletivamente a sincronização de favoritos. A contribuição comprovada desses registros é documental: não há resultados de implantação, indicadores medidos ou execução de sprints a atribuir às propostas.

O plano de testes mantém os casos pendentes e suas datas originais. Assim, não se atribuem ao projeto resultados experimentais, métricas ou responsabilidades executadas sem evidência. A consolidação técnica fornece rastreabilidade para continuidade do desenvolvimento, mas a avaliação das entregas acadêmicas de cada período depende dos enunciados, produtos e diários que documentem sua realização.

Referências

ATIVIDADE PRÁTICA 01: identificando possíveis problemas na manutenção. [s.d.]. Ficha preenchida da disciplina Configuração e Manutenção de Software, Unileste, oitavo período. Documento Word fornecido com a entrega; data não informada. Original preservado em fontes-academicas/pratica-01-preenchida.docx.

ATIVIDADE PRÁTICA 02: identificando possíveis problemas na manutenção. [s.d.]. Ficha preenchida da disciplina Configuração e Manutenção de Software, Unileste, oitavo período. Documento PDF fornecido com a entrega; data não informada. Original preservado em fontes-academicas/pratica-02-preenchida.pdf.

BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. **The Unified Modeling Language User Guide**. 2. ed. Upper Saddle River: Addison-Wesley, 2005. ISBN 9780321267979.

LARMAN, C. **Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo**. 3. ed. Porto Alegre: Bookman, 2007. ISBN 9788560031528.

MATEUS. Alternativas para redução de problemas na manutenção de software. **Engenharia de Software Magazine**, n. 34, 2011. Disponível em: <https://www.devmedia.com.br/alternativas-para-reducao-de-problemas-na-manutencao-de-software-artigo-revista-engenharia-de-software-magazine-34/20113>. Acesso em: 3 nov. 2026.